

Practical Kubernetes Administration and Troubleshooting

Day 3: Networking, Scheduling, and Storage

Instructor: Chad M. Crowell

Day 2 Knowledge Check

8 quick questions before we dive into Day 3

Quiz: Day 2 Recap

1. What is the management hierarchy between a Deployment, a ReplicaSet, and a Pod?
2. In a Deployment's `rollingUpdate` strategy, which field controls the maximum number of pods that can be unavailable during an update?
3. True or False: A failed **readiness** probe causes the container to be restarted.
4. What must be installed in the cluster before a HorizontalPodAutoscaler can scale on CPU utilization?
5. Which `kubectl create` flag generates a Deployment's YAML manifest without actually creating the resource?
6. Are Kubernetes Secrets encrypted at rest by default, or just base64-encoded?
7. Given `failureThreshold: 3`, what does the kubelet do after a pod's liveness probe fails 3 times in a row?
8. Which `kubectl logs` flag shows the logs from a container's previous run after a crash?

Quiz: Answers

1. **Deployment → ReplicaSet → Pod** – the Deployment manages ReplicaSets, and each ReplicaSet manages the Pods
2. `maxUnavailable` – `maxSurge` instead controls how many extra pods can exist above the desired count
3. **False** – a failed readiness probe removes the pod from Service endpoints; it is **not** restarted. Only a failed **liveness** probe triggers a restart
4. **The Metrics Server** – without it, the HPA has no CPU/memory metrics to scale against
5. `--dry-run=client -o yaml` – e.g. `kubectl create deployment nginx --image=nginx:1.27 --dry-run=client -o yaml`
6. **Base64-encoded, not encrypted** – anyone with API access can decode them; enable encryption at rest or use an external secret manager for real protection
7. **The kubelet restarts the container** – liveness failures trigger a restart, unlike readiness failures
8. `kubectl logs <pod-name> --previous`

Day 2 Recap

What We Covered Yesterday

- Namespaces – organizing and isolating cluster resources
- Pods, ReplicaSets, Deployments – the workload resource hierarchy
- Rolling Updates and Rollbacks – zero-downtime deployments
- Scaling – manual and Horizontal Pod Autoscaler
- ConfigMaps and Secrets – externalizing configuration
- Health Checks – liveness, readiness, and startup probes
- Troubleshooting – systematic debugging with kubectl

Where We Left Off

- A running Deployment with health checks and config injection
- Services created with ClusterIP and NodePort
- Ready to go deeper on networking, scheduling, and storage

Day 3 Agenda

Time	Topic
Morning	Kubernetes Networking Model
	Service Types: ClusterIP, NodePort, LoadBalancer
	DNS and CoreDNS
	Network Policies
Afternoon	Scheduling: Taints, Tolerations, Node Affinity
	DaemonSets, Jobs, and CronJobs
	Persistent Volumes and StorageClasses
	Ingress and Gateway API
	Hands-On Labs

Labs Today

- Configure Services and test DNS resolution
- Create a LoadBalancer Service with Linode CCM
- Schedule workloads with taints, tolerations, and node affinity
- Deploy a DaemonSet and a batch Job
- Provision persistent storage using the Linode CSI driver
- Route traffic with an Ingress resource

Kubernetes Networking

Cluster, Pod, and Service networking

The Kubernetes Networking Model

Kubernetes enforces a flat networking model with four rules:

The Four Rules

1. Every **Pod** gets its own unique IP address
2. Containers within a pod share that IP
(communicate via `localhost`)
3. Pods can communicate with **any other pod**
without NAT
4. Nodes can communicate with pods without NAT

The CNI plugin is responsible for implementing this model. Different plugins offer different features (encryption, network policy enforcement, eBPF acceleration).

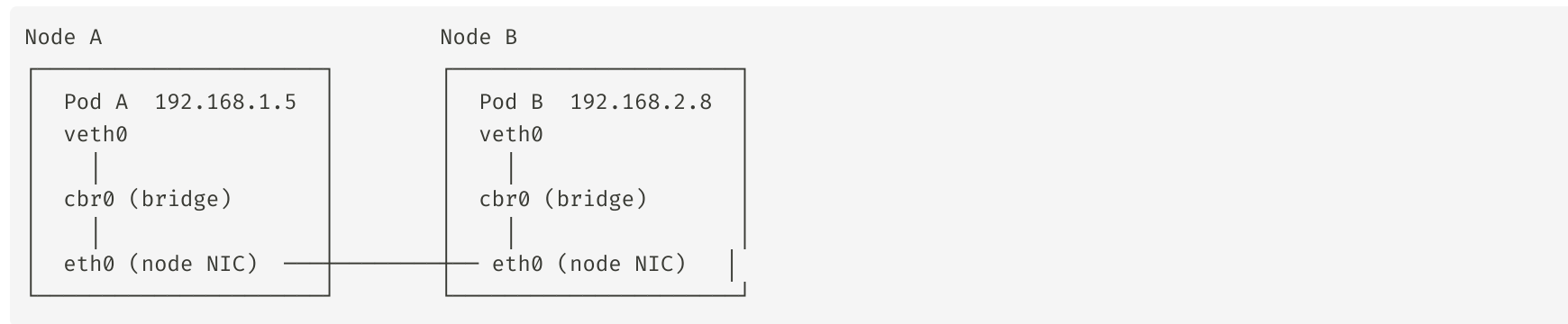
What This Means in Practice

- No port mapping required between pods
- No IP masquerading within the cluster
- The network is **flat** – pods see each other's real IPs
- Enforced by the **CNI plugin** (Calico, Flannel, Cilium, etc.)

Pod Networking

When a pod is created, the CNI plugin:

1. Creates a **virtual network interface** (veth pair) for the pod
2. Assigns an IP from the **pod CIDR** (`192.168.0.0/16` in our cluster)
3. Sets up routing so other pods can reach this IP



Pod-to-pod traffic across nodes is routed through the underlying node network, encapsulated by the CNI plugin (VXLAN, BGP, etc.).

- life of a packet - video

Services

Problem: Pods are ephemeral. Their IPs change when they restart, reschedule, or scale.

Solution: A **Service** provides a stable IP and DNS name that load-balances traffic to a set of pods.

```
Client → Service (stable IP: 10.96.45.12) → Pod A (192.168.1.5)
                                           → Pod B (192.168.1.6)
                                           → Pod C (192.168.2.8)
```

How Services Select Pods

Services use **label selectors** – the same labels you put on pods:

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web          # matches pods with this label
  ports:
    - port: 80
      targetPort: 8080
```

Service Types

Type	Accessibility	Use Case
ClusterIP	Inside the cluster only	Internal service-to-service communication
NodePort	Cluster + external via <code><NodeIP>:<Port></code>	Development, direct node access
LoadBalancer	External via cloud load balancer	Production external traffic
ExternalName	DNS alias to external service	Migrate external services into k8s DNS

ClusterIP

The default Service type. Allocates a stable **virtual IP** reachable only from within the cluster.

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP      # default – can be omitted
  selector:
    app: web
  ports:
    - port: 80          # port clients connect to
      targetPort: 80    # port the container listens on
```

```
# kube-proxy programs iptables/IPVS rules on every node
```

```
kubectl get svc web
```

# NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
# web	ClusterIP	10.96.45.12	<none>	80/TCP	1m

kube-proxy watches the API server for Service and Endpoint changes, then programs iptables/IPVS rules on every node to redirect traffic to the correct pod IPs.

NodePort

Exposes the Service on a **static port** on every node's IP. Traffic to `<NodeIP>:<NodePort>` is forwarded to the Service.

```
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80          # cluster-internal port
      targetPort: 80    # container port
      nodePort: 30080   # external port (30000-32767), or omit to auto-assign
```

```
kubectl get svc web-nodeport
```

# NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
# web-nodeport	NodePort	10.96.45.13	<none>	80:30080/TCP	1m

```
# access from outside the cluster
```

```
curl http://<any-node-ip>:30080
```

NodePort works but is not production-grade for external traffic – use LoadBalancer or Ingress instead.

LoadBalancer

Provisions an **external load balancer** from the cloud provider and assigns a public IP.

```
apiVersion: v1
kind: Service
metadata:
  name: web-lb
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

```
kubectl get svc web-lb
```

#	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
#	web-lb	LoadBalancer	10.96.45.14	45.79.123.45	80:31204/TCP	2m

```
# access from the internet
```

```
curl http://45.79.123.45
```

The external IP is assigned by the **Cloud Controller Manager (CCM)**. On bare-metal clusters, use MetalLB to get this functionality.

Install CCM

```
kubectl create secret generic linode \  
  --namespace kube-system \  
  --from-literal=apiToken=$LINODE_API_TOKEN \  
  --from-literal=region=us-ord  
  
# Install Helm  
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash  
  
# install CCM  
helm repo add linode-cloud-controller-manager https://linode.github.io/linode-cloud-controller-manager/  
helm repo update  
  
helm upgrade --install ccm-linode \  
  linode-cloud-controller-manager/ccm-linode \  
  --namespace kube-system \  
  --set secretRef.name=linode
```

ExternalName

Maps a Service to an **external DNS name** – no proxying, just a CNAME.

```
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  type: ExternalName
  externalName: my-database.rds.amazonaws.com
```

Pods that connect to `db` within the cluster will resolve to `my-database.rds.amazonaws.com`.

Use Cases

- Abstract a third-party database behind a stable in-cluster name
- Environment-specific routing (different external host per environment)
- Migrate external services into the cluster DNS without changing app config

Endpoints and Troubleshooting

When you create a Service with a selector, Kubernetes automatically creates an **Endpoints** object listing the IPs of matching pods.

```
# see the pod IPs behind a service
kubectl get endpoints web
# NAME      ENDPOINTS                                     AGE
# web       192.168.1.5:8080,192.168.1.6:8080,192.168.2.8:8080 5m
```

If Endpoints Is Empty

```
# endpoints empty = selector doesn't match any pods
kubectl get endpoints web
# NAME      ENDPOINTS    AGE
# web       <none>       1m

# diagnose:
kubectl describe svc web | grep Selector
kubectl get pods --show-labels
# make sure pod labels match the service selector exactly
```

DNS and CoreDNS

How Kubernetes resolves service names

Kubernetes DNS

Every Service gets a DNS name automatically. Pods use this to find each other **without hardcoding IPs**.

DNS Name Format

```
<service-name>.<namespace>.svc.cluster.local
```

Example	Works From
<code>web</code>	Same namespace only
<code>web.default</code>	Any namespace
<code>web.default.svc.cluster.local</code>	Always (fully qualified)

```
# test DNS from inside a pod
kubectl run dns-test --image=busybox:1.26 --rm -it --restart=Never -- \
  nslookup web.default.svc.cluster.local
```

CoreDNS

CoreDNS is the DNS server that runs as a Deployment in `kube-system`. Every pod is configured to use it as its nameserver.

```
# CoreDNS runs as a Deployment
kubectl get deployment coredns -n kube-system

# and exposes a Service
kubectl get svc kube-dns -n kube-system

# NAME          TYPE          CLUSTER-IP    PORT(S)
# kube-dns      ClusterIP     10.96.0.10     53/UDP,53/TCP

# each pod gets this injected into /etc/resolv.conf
kubectl exec -it <pod-name> -- cat /etc/resolv.conf
# nameserver 10.96.0.10
# search default.svc.cluster.local svc.cluster.local cluster.local
```

The `search` domains allow short names like `web` to resolve – the resolver tries `web.default.svc.cluster.local` automatically.

CoreDNS Configuration

CoreDNS is configured via a **ConfigMap** in `kube-system` :

```
kubectl get configmap coredns -n kube-system -o yaml
```

```
.:53 {
  errors
  health { lameduck 5s }
  ready
  kubernetes cluster.local in-addr.arpa ip6.arpa {
    pods insecure
    fallthrough in-addr.arpa ip6.arpa
    ttl 30
  }
  prometheus :9153
  forward . /etc/resolv.conf    # external DNS forwarded to node resolver
  cache 30
  loop
  reload
  loadbalance
}
```

DNS Troubleshooting

```
# test from inside a pod
kubectl run dns-test --image=busybox:1.26 --rm -it --restart=Never -- sh

# inside the pod:
nslookup kubernetes.default          # should resolve to 10.96.0.1
nslookup web.default.svc.cluster.local
nslookup google.com                  # external DNS (forwarded by CoreDNS)

# check CoreDNS logs
kubectl logs -n kube-system -l k8s-app=kube-dns

# check CoreDNS pods are running
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

Common DNS Problems

Symptom	Likely Cause
<code>nslookup web</code> fails	Wrong namespace, or service doesn't exist
<code>nslookup google.com</code> fails	CoreDNS forward config, node DNS issue

Network Policies

Controlling pod-to-pod traffic

What Are Network Policies?

By default, **all pods can talk to all other pods**. Network Policies let you restrict traffic at the pod level.

Without Network Policy:	With Network Policy:
frontend → backend	frontend → backend ✓
attacker → backend	attacker → backend ✗

Requirements

- Your **CNI plugin must support** Network Policies (Calico ✓, Cilium ✓, Flannel ✗)
- Policies are **additive** – if any policy selects a pod, only allowed traffic passes
- A pod with **no policy** is fully open (allow all)

Network Policy Example

Allow `frontend` pods to reach `backend` on port 8080, deny everything else:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: dev
spec:
  podSelector:
    matchLabels:
      app: backend          # this policy applies to backend pods
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: web      # only allow traffic from pods with web label
  ports:
    - protocol: TCP
      port: 3306
```

Deny-All Pattern

A common baseline: **deny all ingress by default**, then explicitly allow what's needed.

```
# deny all ingress to all pods in this namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
spec:
  podSelector: {} # applies to ALL pods
  policyTypes:
    - Ingress
  # no ingress rules = deny all
```

Test connectivity

```
kubectl apply -f netpol.yaml
kubectl get networkpolicy

# create backend service backend.dev.svc.cluster.local
kubectl expose po backend --port 3306

# run web container with label app=web
kubectl run mysql-web --rm -it --image=mysql:8 -n dev --labels="app=web" --env="MYSQL_ROOT_PASSWORD=changeme" -- bash

# from a shell inside the container
mysql -h backend.dev.svc.cluster.local -P 3306 -u root -pSuperSecretP@ssword! -e "SELECT 1;"
```

Scheduling and Workload Distribution

Controlling where pods run

How the Kubernetes Scheduler Works

The **kube-scheduler** watches for unscheduled pods and assigns them to nodes in two phases:

Phase 1: Filtering

Remove nodes that don't meet requirements:

- Not enough CPU or memory
- Has a taint the pod doesn't tolerate
- Missing a required node label
- Volume not available in that zone

Phase 2: Scoring

Rank remaining nodes and pick the best:

- Prefer nodes with more free resources
- Prefer nodes that already have the required image cached
- Honor any affinity preferences

Node Selectors

The simplest way to constrain a pod to nodes with a specific label.

Label a Node

```
kubectl label node worker-1 disktype=ssd
kubectl get nodes --show-labels
```

Use nodeSelector in a Pod

```
spec:
  nodeSelector:
    disktype: ssd    # only schedule on nodes with this label
  containers:
  - name: app
    image: myapp:latest
```

`nodeSelector` is simple but inflexible – it's an exact match with no fallback. Use **Node Affinity** for more expressive rules.

Node Affinity

Node Affinity is the expressive replacement for `nodeSelector` with required and preferred rules.

Field	Meaning
<code>requiredDuring ...</code>	Hard rule – pod won't schedule if no match
<code>preferredDuring ...</code>	Soft rule – scheduler tries to honor it

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disktype
                operator: In
                values:
                  - ssd
                  - nvme
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 50
          preference:
            matchExpressions:
              - key: zone
                operator: In
                values:
                  - us-east-1a
```

Taints and Tolerations

Taints are placed on **nodes** to repel pods. **Tolerations** are placed on **pods** to allow them onto tainted nodes.

Taint a Node

```
# syntax: key=value:effect
kubectl taint node worker-2 gpu=true:NoSchedule

# remove the taint
kubectl taint node worker-2 gpu=true:NoSchedule-
```

Tolerate the Taint in a Pod

```
spec:
  tolerations:
  - key: "gpu"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
  containers:
  - name: gpu-app
    image: nvidia/cuda:12.0
```

Taint Effects

Effect	Behavior
NoSchedule	New pods without the toleration won't be scheduled here
PreferNoSchedule	Scheduler tries to avoid this node but isn't forced to
NoExecute	Existing pods without the toleration are evicted ; new pods won't schedule

Built-In Taints

```
# control plane nodes are tainted by default
kubectl describe node control-plane | grep Taint
# Taints: node-role.kubernetes.io/control-plane:NoSchedule

# nodes that are NotReady are automatically tainted
# node.kubernetes.io/not-ready:NoExecute (auto-applied by node lifecycle controller)
```

Worker nodes that join the cluster have no taints by default – all pods can land on them.

Pod Affinity and Anti-Affinity

Schedule pods **relative to other pods** – co-locate them or spread them apart.

```
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: web
          topologyKey: "kubernetes.io/hostname"
        # don't place two web pods on the same node
```

```
podAffinity:
  preferredDuringSchedulingIgnoredDuringExecution:
    - weight: 100
      podAffinityTerm:
        labelSelector:
          matchLabels:
            app: cache
        topologyKey: "kubernetes.io/hostname"
      # prefer placing this pod on the same node as cache pods
```

topologyKey: kubernetes.io/hostname = per-node spread. topology.kubernetes.io/zone = per-zone spread.

Manual Scheduling

If you want to bypass the scheduler entirely, set `nodeName` directly:

```
spec:
  nodeName: worker-1    # skip scheduler, place directly on this node
  containers:
  - name: app
    image: nginx:1.27
```

Debugging Scheduling Failures

```
# check which node a pod landed on
kubectl get pod <name> -o wide

# see why a pod is Pending
kubectl describe pod <name> | grep -A10 Events

# common messages:
# "0/2 nodes available: 1 Insufficient cpu, 1 node(s) had taint..."
# "0/2 nodes available: 2 node(s) didn't match node selector"
```

DaemonSets, Jobs, and CronJobs

Beyond Deployments

DaemonSets

A **DaemonSet** ensures that **one copy of a pod runs on every node**. As nodes are added, pods are added. As nodes are removed, pods are garbage collected.

Common Use Cases

- **Log collectors** – Fluentd, Fluent Bit on every node
- **Monitoring agents** – Prometheus node-exporter, Datadog agent
- **Networking** – CNI plugins, kube-proxy itself is a DaemonSet
- **Storage** – CSI node drivers

```
# DaemonSets running in kube-system
kubectl get daemonsets -n kube-system
```

# NAME	DESIRED	CURRENT	READY	UP-TO-DATE	NODE SELECTOR
# calico-node	2	2	2	2	kubernetes.io/os=linux
# kube-proxy	2	2	2	2	kubernetes.io/os=linux

DaemonSet YAML

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          effect: NoSchedule      # also run on control plane nodes
      containers:
        - name: node-exporter
          image: prom/node-exporter:latest
          ports:
            - containerPort: 9100
              hostPort: 9100
```

Jobs

A **Job** creates one or more pods and ensures they **run to completion**. Unlike a Deployment, a Job does not restart successfully completed pods.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate-demo
spec:
  completions: 1
  parallelism: 1
  backoffLimit: 3
  ttlSecondsAfterFinished: 300
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: migrate
        image: python:3.10.12
        command:
        - python
        - -c
        - |
          print("Running demo migration")
          print("Applying migration 0001_initial")
```

Job Patterns

Pattern	completions	parallelism	Use Case
Single run	1	1	One-off task
Fixed completion count	N	1	Process N items sequentially
Work queue	N	N	Process N items in parallel

```
# parallel job: process 5 items with 2 workers at a time
spec:
  completions: 5
  parallelism: 2
```

Jobs are useful for database migrations, batch processing, report generation, and one-time initialization tasks.

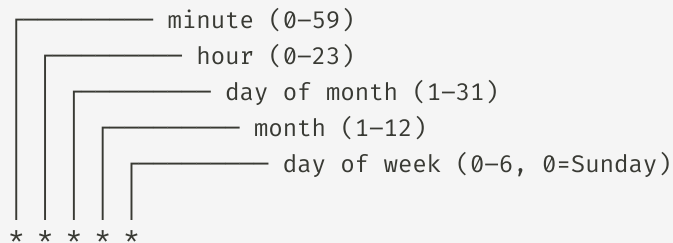
CronJobs

A **CronJob** creates Jobs on a **schedule** using standard cron syntax.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup
spec:
  schedule: "0 2 * * *"      # run at 2:00 AM every day
  concurrencyPolicy: Forbid   # don't run if previous is still running
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: backup-tool:latest
              command: ["/backup.sh"]
```

```
kubectl get cronjobs
kubectl get jobs      # shows jobs created by the cronjob
```

Cron Schedule Syntax



Schedule	Meaning
<code>0 * * * *</code>	Every hour at <00 />
<code>* / 15 * * * *</code>	Every 15 minutes
<code>0 2 * * *</code>	Daily at 2:00 AM
<code>0 2 * * 0</code>	Weekly on Sunday at 2:00 AM
<code>0 2 1 * *</code>	Monthly on the 1st at 2:00 AM

ConcurrencyPolicy options: Allow (default), Forbid (skip if previous is running), Replace (kill previous, start new).

Persistent Storage in Kubernetes

Volumes, PVs, PVCs, and StorageClasses

Why Persistent Storage?

Containers are ephemeral – when a pod restarts, **all data written to the container filesystem is lost.**

Pod restarts → Container filesystem wiped → Database data gone!

The Solution: Volumes

Type	Lifecycle	Use Case
<code>emptyDir</code>	Pod lifetime	Shared scratch space between containers
<code>hostPath</code>	Node lifetime	Access node filesystem (log collectors)
<code>PersistentVolumeClaim</code>	Independent	Databases, stateful apps
<code>configMap</code> / <code>secret</code>	Config lifetime	Inject configuration files
<code>nfs</code>	External server	Shared read-write across pods

emptyDir and hostPath

emptyDir – Shared Scratch Space
Between Containers in a Pod

```
spec:
  volumes:
    - name: shared-data
      emptyDir: {}
  containers:
    - name: writer
      image: busybox
      volumeMounts:
        - name: shared-data
          mountPath: /data
    - name: reader
      image: busybox
      volumeMounts:
        - name: shared-data
          mountPath: /data
```

hostPath – Mount a Node Directory

```
volumes:
  - name: host-logs
    hostPath:
      path: /var/log
      type: Directory
```

PersistentVolumes (PV)

A **PersistentVolume** is a piece of storage provisioned by an admin (or dynamically by a StorageClass). It exists **independently of any pod**.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-manual
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /mnt/data    # use a real provisioner in product
```

Access Modes

Mode	Abbreviation	Meaning
ReadWriteOnce	RWO	One node can read and write
ReadOnlyMany	ROX	Many nodes can read
ReadWriteMany	RWX	Many nodes can read and write

PersistentVolumeClaims (PVC)

A **PersistentVolumeClaim** is a request for storage by a pod. Kubernetes binds the PVC to a matching PV.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 100Mi
  storageClassName: linode-block-storage-retain
```

```
kubectl get pvc
```

#	NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
#	my-pvc	Bound	pv-manual	10Gi	RWO	manual	30s

Using a PVC in a Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: db
spec:
  containers:
    - name: postgres
      image: postgres:16
      env:
        - name: POSTGRES_PASSWORD
          value: mysecretpassword
        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata
      ports:
        - containerPort: 5432
          name: postgres
      volumeMounts:
        - name: db-storage
          mountPath: /var/lib/postgresql/data
  volumes:
    - name: db-storage
      persistentVolumeClaim:
        claimName: my-pvc    # reference the PVC by name
```

StorageClasses

A **StorageClass** describes a type of storage and enables **dynamic provisioning** – Kubernetes creates PVs automatically when a PVC requests it.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: linode-block-storage
provisioner: linodebs.csi.linode.com
parameters:
  type: standard
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer # delay binding until pod is scheduled
allowVolumeExpansion: true
```

```
kubectl get storageclasses
```

# NAME	PROVISIONER	RECLAIM POLICY
# linode-block-storage	linodebs.csi.linode.com	Delete

Static vs Dynamic Provisioning

Static Provisioning

Admin manually creates PVs ahead of time:

```
Admin creates PV (10Gi)
↓
Developer creates PVC (5Gi)
↓
Kubernetes binds PVC to PV
(PVC gets 10Gi, wastes 5Gi)
```

- More admin control
- Admin bottleneck
- Storage waste possible

Dynamic Provisioning

StorageClass provisions PVs on demand:

```
Developer creates PVC (5Gi)
↓
StorageClass provisions
exact 5Gi volume
↓
PV created and PVC bound
```

- Developer self-service
- No waste
- Requires CSI driver

Linode Block Storage CSI Driver

The **Linode CSI driver** allows Kubernetes to dynamically provision Linode Block Storage volumes.

Install the CSI Driver

```
export REGION=us-ord

kubectl delete secret linode -n kube-system

helm repo add linode-csi https://linode.github.io/linode-blockstorage-csi-driver/
helm repo update linode-csi

helm install linode-csi-driver \
  --namespace kube-system \
  --set apiToken="${LINODE_API_TOKEN}" \
  --set region="${REGION}" \
  linode-csi/linode-blockstorage-csi-driver
```

Dynamic PVC using the StorageClass

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-storage
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: linode-block-storage
```

```
# Pending → Bound once a pod schedules and triggers provisioning
kubectl get pvc app-storage -w
```

PVC Reclaim Policy

When a PVC is deleted, what happens to the underlying volume?

Policy	Behavior
Delete	PV and the underlying storage are deleted (cloud default)
Retain	PV remains – admin must manually reclaim or delete
Recycle	Deprecated – basic scrub then made available again

```
# check a PV's reclaim policy
kubectl get pv
# NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM
# pv-abc        10Gi      RWO           Delete          Bound   default/app-storage

# after PVC is deleted with Retain: PV goes to Released state
# Released = not available for new claims until manually reclaimed
```

Use `Retain` for databases in production – accidental PVC deletion won't destroy your data.

Ingress and Gateway API

Routing external HTTP/HTTPS traffic into the cluster

Why Ingress?

Problem with LoadBalancer Services: Each Service requires its own cloud load balancer – expensive at scale.

Without Ingress:

LoadBalancer → Service A (web)	\$ per load balancer
LoadBalancer → Service B (api)	\$ per load balancer
LoadBalancer → Service C (docs)	\$ per load balancer

With Ingress:

One LoadBalancer → Ingress Controller → /	→ Service A
→ /api	→ Service B
→ /docs	→ Service C

Ingress gives you:

- HTTP/HTTPS routing based on **host** and **path**
- **TLS termination** in one place
- Single external IP for multiple services

Ingress Controllers

An **Ingress resource** is just configuration – you need an **Ingress Controller** to act on it.

Controller	Notes
ingress-nginx	Most common, open source, feature-rich
Traefik	Cloud-native, automatic TLS via Let's Encrypt
HAProxy Ingress	High performance, enterprise features
AWS Load Balancer Controller	Native AWS ALB integration

Install ingress-nginx

```
# Install Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Create namespace
kubectl create namespace ingress-nginx
```

Ingress Resource

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: test-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: mydomain.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

 gist.github.com/chadmcrowell/f987c7a646fed3fe6c6ed9fabcb21df

Ingress with TLS

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress-tls
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - myapp.example.com
      secretName: myapp-tls      # TLS cert stored as a Secret
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web
            port:
              number: 80
```

```
# create the TLS secret from cert files
kubectl create secret tls myapp-tls --cert=tls.crt --key=tls.key
```

In production, use **cert-manager** to automatically provision and renew Let's Encrypt certificates.

Gateway API

Gateway API is the next-generation Kubernetes networking API, designed to replace Ingress with a more expressive model.

Why Gateway API over Ingress?

	Ingress	Gateway API
Routing rules	Host + path only	Host, path, headers, methods, weights
Protocol support	HTTP/HTTPS	HTTP, HTTPS, TCP, UDP, gRPC
Role separation	Single resource	GatewayClass, Gateway, Routes (multi-team)
Extensibility	Annotations (vendor-specific)	First-class spec
Status	Stable (legacy)	GA since Kubernetes 1.31

Gateway API Resources

Infrastructure Team

GatewayClass
(which controller to use)

Gateway
(listener: port 80, 443)

Developer Team

HTTPRoute
(HTTP routing rules)

TCPRoute
(TCP routing rules)

UDPRoute
(UDP routing rules)

Install the Gateway API CRDs

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.2.1/standard-install.yaml
```

```
kubectl get crd | grep gateway  
# gatewayclasses.gateway.networking.k8s.io  
# gateways.gateway.networking.k8s.io  
# httproutes.gateway.networking.k8s.io
```

GatewayClass and Gateway

```
# GatewayClass – cluster-scoped, managed by infrastructure team
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx
spec:
  controllerName: k8s.nginx.org/nginx-gateway-controller
```

```
# Gateway – defines what ports and protocols to listen on
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: prod-gateway
  namespace: infra
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      port: 80
      protocol: HTTP
      allowedRoutes:
        namespaces:
          from: All    # allow HTTPRoutes from any namespace
```

HTTPRoute

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web-route
  namespace: default
spec:
  parentRefs:
    - name: prod-gateway
      namespace: infra
  hostnames:
    ...
```

 gist.github.com/chadmcrowell/0f3ec25d2fe9d878b7d7a04b46178cfb

HTTPRoute – Advanced Traffic Management

Traffic splitting for canary deployments:

```
rules:
- backendRefs:
  - name: web-v1
    port: 80
    weight: 90    # 90% of traffic to stable version
  - name: web-v2
    port: 80
    weight: 10    # 10% of traffic to canary
```

Header-based routing (route beta users to new version):

```
rules:
- matches:
  - headers:
    - name: x-user-group
      value: beta
  backendRefs:
  - name: web-beta
    port: 80
```

Traffic splitting and header routing required custom annotations with Ingress – with Gateway API it's part of the standard spec.

Linode Cloud Controller Manager (CCM)

The **Linode CCM** integrates Kubernetes with Linode's NodeBalancer service. When you create a **LoadBalancer** Service, the CCM provisions a Linode NodeBalancer automatically.

```
# CCM runs as a Deployment on control plane nodes
kubectl get pods -n kube-system | grep ccm

# create a LoadBalancer Service
kubectl expose deployment web --type=LoadBalancer --port=80

# CCM provisions a NodeBalancer and assigns an external IP (~60s)
kubectl get svc web -w
```

#	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
#	web	LoadBalancer	10.96.45.14	170.187.xxx.xx	80:31204/TCP

```
# access from the internet
curl http://170.187.xxx.xx
```

```
# if external IP is stuck in <pending>, check CCM logs
kubectl logs -n kube-system -l app=linode-ccm
```

Hands-On Labs

Networking, Scheduling, Storage, and Ingress

Lab 1: Services and DNS

Deploy an Application

```
kubectl create deployment web --image=nginx:1.27 --replicas=3
kubectl expose deployment web --port=80 --name=web-clusterip
```

Test DNS Resolution

```
kubectl run dns-test --image=busybox:1.26 --rm -it --restart=Never -- \
  nslookup web-clusterip.default.svc.cluster.local
```

Verify Endpoints

```
kubectl get endpoints web-clusterip
kubectl describe svc web-clusterip
```

Test Connectivity

```
kubectl run curl-test --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://web-clusterip.default.svc.cluster.local
```

Lab 2: NodePort and LoadBalancer

Create a NodePort Service

```
kubectl expose deployment web --port=80 --type=NodePort --name=web-nodeport
kubectl get svc web-nodeport

# test from outside the cluster using any node's IP
curl http://<any-node-ip>:<nodeport>
```

Create a LoadBalancer Service (Linode CCM)

```
kubectl expose deployment web --port=80 --type=LoadBalancer --name=web-lb

# watch for external IP assignment (~60s)
kubectl get svc web-lb -w

# test from the internet
curl http://<external-ip>
```

Clean Up

```
kubectl delete svc web-nodeport web-lb
```

Lab 3: Scheduling with Taints and Tolerations

Label and Taint Nodes

```
kubectl label node worker-1 disktype=ssd
kubectl taint node worker-2 dedicated=gpu:NoSchedule
```

Deploy with nodeSelector

```
kubectl create deployment ssd-app --image=nginx:1.27 --replicas=3 \
  --dry-run=client -o yaml > ssd-app.yaml
```

Edit `ssd-app.yaml` and add under `spec.template.spec` :

```
nodeSelector:
  disktype: ssd
```

```
kubectl apply -f ssd-app.yaml
kubectl get pods -o wide    # all pods should land on worker-1
```

Lab 3: Scheduling (cont.)

Deploy with Toleration for Tainted Node

```
kubectl create deployment gpu-app --image=nginx:1.27 \
--dry-run=client -o yaml > gpu-app.yaml
```

Edit `gpu-app.yaml` and add under `spec.template.spec` :

```
tolerations:
- key: "dedicated"
  operator: "Equal"
  value: "gpu"
  effect: "NoSchedule"
```

```
kubectl apply -f gpu-app.yaml
kubectl get pods -o wide    # pod can now schedule on worker-2
```

Remove the Taint

```
kubectl taint node worker-2 dedicated=gpu:NoSchedule-
kubectl delete deployment ssd-app gpu-app
```

Lab 4: DaemonSet and CronJob

Deploy a DaemonSet

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-logger
spec:
  selector:
    matchLabels:
      app: node-logger
  template:
    metadata:
      ...
```

 gist.github.com/chadmcrowell/7317bc753d0df77ca9e1a7b4042d69fd

```
# one pod per node
kubectl get pods -o wide -l app=node-logger
```

Lab 4: CronJob

Create a CronJob

```
kubectl create cronjob hello \  
  --image=busybox \  
  --schedule="*/1 * * * *" \  
  -- sh -c 'echo "Hello at $(date)">'
```

watch jobs being created every minute

```
kubectl get jobs -w
```

view logs from the most recently created job

```
kubectl logs $(kubectl get pods -l "job-name" --sort-by=.metadata.creationTimestamp -o name | tail -1)
```

Clean Up

```
kubectl delete cronjob hello  
kubectl delete daemonset node-logger
```

Lab 5: Persistent Storage with PVC

Create a PVC

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: linode-block-storage
EOF

kubectl get pvc data-pvc    # Pending → Bound after pod uses it
```

Lab 5: Persistent Storage (cont.)

Deploy a Pod that Uses the PVC

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: storage-demo
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "echo 'hello persistent world' > /data/test.txt"]
    volumeMounts:
    - name: data
      mountPath: /data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: data-pvc
EOF
```

```
kubectl get pvc data-pvc # should be Bound
kubectl exec storage-demo -- cat /data/test.txt
```

Delete and Recreate – Data Persists

```
kubectl delete pod storage-demo
kubectl apply -f storage-demo.yaml
kubectl exec storage-demo -- cat /data/test.txt # data s
```

Lab 6: Ingress

Deploy Two Services

```
kubectl create deployment web --image=nginx:1.27
kubectl expose deployment web --port=80 --name=web-svc

kubectl create deployment api --image=hashicorp/http-echo
  -- /http-echo -text="hello from api"
kubectl expose deployment api --port=5678 --name=api-svc
```

Create an Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: demo-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - http:
      paths:
    ...
```



gist.github.com/chadmcrowell/c8fcf57d994d97cab0f336046635aa22

Lab 6: Ingress (cont.)

Test the Ingress

```
# get the ingress controller's external IP
kubectl get svc -n ingress-nginx ingress-nginx-controller

# test path routing
curl http://<ingress-ip>/
curl http://<ingress-ip>/api

# inspect
kubectl describe ingress demo-ingress
kubectl get ingress
```

Full Clean Up

```
kubectl delete ingress demo-ingress
kubectl delete svc web-svc api-svc
kubectl delete deployment web api
kubectl delete pod storage-demo
kubectl delete pvc data-pvc
```

Day 3 Recap

What We Covered Today

-  Kubernetes Networking Model – flat network, pod IPs, CNI
-  Service Types – ClusterIP, NodePort, LoadBalancer, ExternalName
-  DNS and CoreDNS – service discovery and name resolution
-  Network Policies – restricting pod-to-pod traffic with Calico
-  Scheduling – node selectors, affinity, taints and tolerations
-  DaemonSets – one pod per node for system-level tasks
-  Jobs and CronJobs – batch workloads and scheduled tasks
-  Persistent Storage – PVs, PVCs, StorageClasses, Linode CSI
-  Ingress – HTTP routing with ingress-nginx, TLS termination
-  Gateway API – HTTPRoute, traffic splitting, header routing
-  Linode CCM – automatic NodeBalancer provisioning

Coming Up Tomorrow

Day 4: Security, Monitoring, and Troubleshooting

- **Security Essentials** – RBAC, Cluster Roles, Service Accounts, Security Contexts, Network Policies
- **Monitoring and Observability** – Prometheus, Grafana, Alertmanager
- **Helm** – Package management for Kubernetes
- **Troubleshooting and Cluster Operations** – Diagnosing node and pod issues, cluster upgrades, etcd backup and restore

Labs

- RBAC and access control exercises
- Deploy a full monitoring stack with Helm
- Multi-node cluster operations and validation
- Cluster upgrade procedure (control plane + workers)
- etcd snapshot backup and restore

Questions?